

客户端笔记

Alfie

2025 年 10 月 21 日

目录

1	Android 操作系统介绍	1
2	Android UI 技术	5
3	终端网络技术	8
4	终端存储技术	11
5	Android 安全开发	13
6	多终端平台技术	15
7	iOS UI 技术	17

1 Android 操作系统介绍

Day 1 2025/9/24

移动应用开发基础结构：

- 交互界面
- 网路协议
- 数据存储
- 终端安全
- 性能，稳定性
- 多媒体（音频，视频）
- 外设前沿技术（手表）

开发工具 -> Android Studio

移动设备软件系统平台：

- Android
- iOS
- wp

简单设计：

- 登录模块（登录，注册）
- 主界面（呈现聊天列表）
- 聊天功能（对话）
- 消息推送（消息的送达）

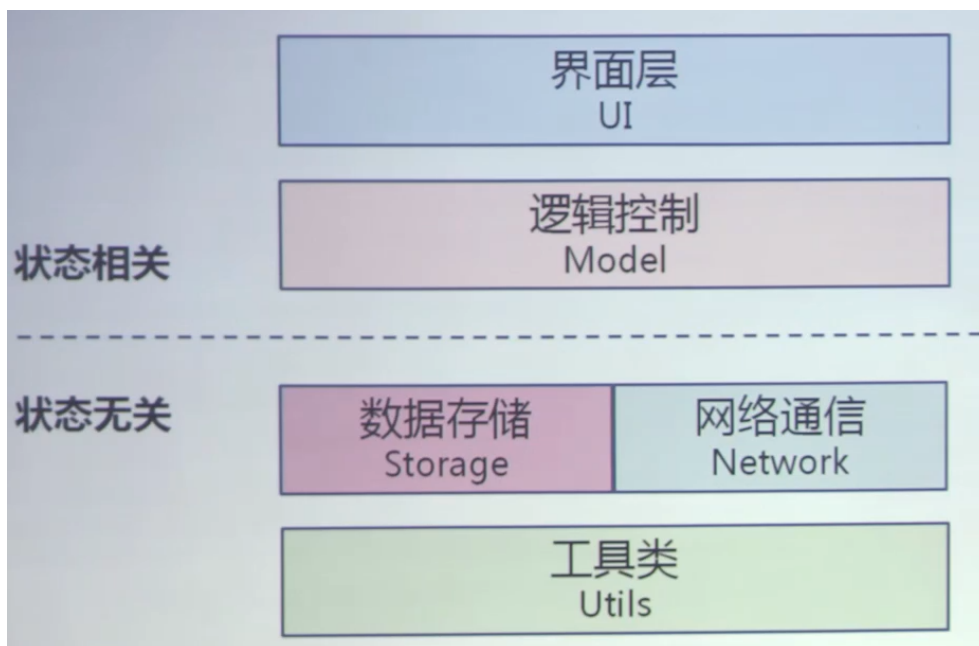


图 3: 微信的软件分层

实用工具与技巧：

- 界面层-> 继承，界面库
- 工具类-> 积累工具包（无状态，可复用，单一职责）
- 存储-> SQLite, ORM 机制（减少人工写入代码成本）
- 网络层 -> protobuf/json/xml, 数据加密, socket vs http

注重分层设计，关注不同层级的设计与开发重点

Android 四大组件：

- Activity
- Service
- BroadcastReceiver
- ContentProvider

Android 基于 Linux 开发，软件以 APK 作为载体，程序生命周期重新通过四大组件进行封装。

多进程：

- 高内聚，低耦合
- 逻辑清晰，进程独立互不影响
- 性能消耗增加：CPU，内存
- 引入复杂度和稳定性问题

模块化开发：

- all-in-one 单一 app，内部进行分层设计和横向模块划分
- 组合 App 形式
- 单一 app 形式，但内部模块独立开发编译发布
- 开发质量管理-> 自动代码生成，动态注解机制，持续集成系统，静态代码扫描，单元测试（逻辑层为主），稳定性监控

2 Android UI 技术

Day 2 2025/9/26

资源

资源 (Resource) 是从代码中分离，用于 UI 呈现和存储其他逻辑数据的静态内容

- icon 图标
- MP3 文件
- txt 文件
- XML 文件

资源的组织形式

- assets 目录：原始资源，任意格式和任意组织方式（音频，视频），不做编译，只做迁移
- res 目录：系统化资源，系统统一组织管理

资源编译的意义

- 使用索引方式提高读取速度
- 编译成二进制提高解析速度
- 减少资源的大小（裸图片的压缩）

目录	资源类型
animator/ anim/	属性动画；补间动画
color/	颜色状态Selector
drawable/	Bitmap 或 XML Drawable files
layout/	UI布局
menu/	应用菜单
values/	包含颜色值、字符串、数组等各种常量定义的资源文件
xml/	XML属性配置文件
raw/	保存原始格式文件，如音频视频等

图 4: res 类型

适配类型	配置(configuration)	限定符(Qualifier)
本地化	MCC & MNC	mcc310
	Language & Region	en、en-rUS
屏幕差异	Layout Direction/Smallest Width/Available width/ Available Height/Screen Size/Screen Aspect	
	Screen Orientation	port、land
UI模式	UI Mode / Night Mode	
像素密度	Screen Pixel Density	ldpi、xhdpi、xxhdpi、nodpi
输入设备	Touchscreen Type / Keyboard Availability / Primary Text Input Method Navigation Key Availability / Primary Non-touch Navigation Method	
系统版本	Platform Version (API Level)	v3、v4、v7

图 5: 资源的适配原则

UI (User Interface)

- 窗口
- 控件 (View)
- 消息循环机制: UI 的渲染是单线程的

动画

- 定时器 + 分时渲染 (效果一般)
- Animations: 实现基本上所有动画效果-> 帧动画, 补间动画, 属性动画

UI 评估指标

- 最佳的 UI 流畅体验: 一秒钟 60 帧 -> 在 16ms 内准备好下一帧
- UI 性能优化-> 避免过度绘制 (单区域尽量只做一次绘制), 优化 layout 减少层级 (减少递归调用), 主线程不做耗时操作 (不做 IO 操作, 不做网络操作, 不做过多 UI 布局计算)

3 终端网络技术

Day 3 2025/9/29

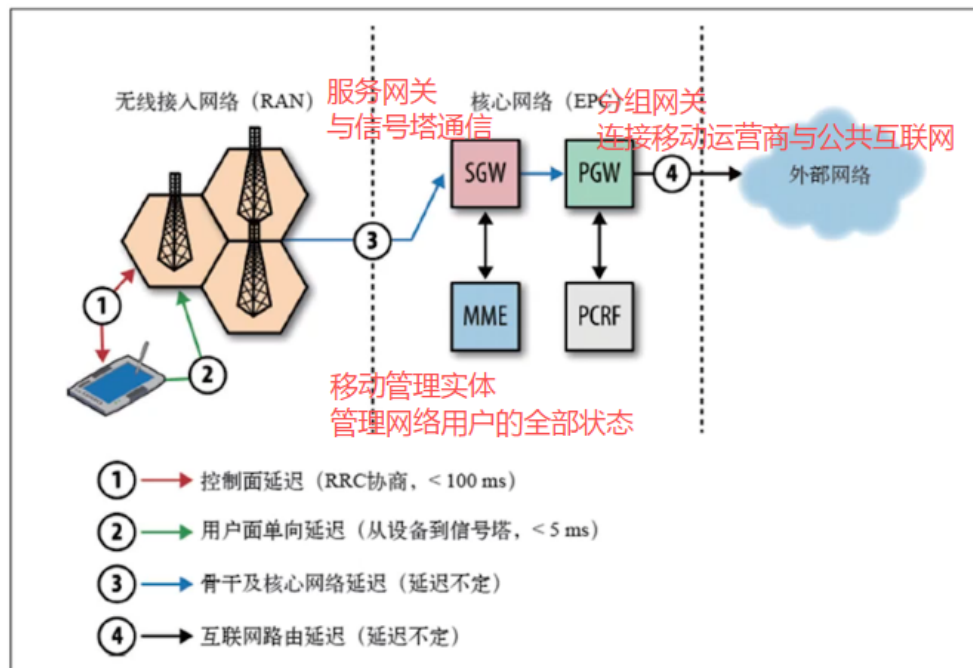


图 6: 无线网络 LTE 物理层架构

随着无线网络的发展，网络的带宽和延迟都得到了改善。移动网络整个传输过程只有手机至 RAN 信号塔是无线的，这个过程极不稳定，丢包率高。无线网络的另一个特点是带宽受限高延迟，并可能受到第三方影响

影响无线传输的因素：

- 空气微尘
- 温度
- 湿度
- 障碍物

- 基站拥挤
- 信号盲点
- 高速移动

无线网络的特点：

- 为多代并存而设计
- 移动网络的高度可变性
- 移动设备的资源约束
- TCP 设计针对有线网络，并不适用于无线网络编程，但它又是事实上的标准

TCP 设计：

- 可靠——序号与确认，检验和（2 Byte），超时重传
- 流量控制和拥塞控制——滑动窗口，慢开始和拥塞避免

服务端实践：

- 采用 WestWood+ 算法
- 增大 TCP 的初始拥塞窗口（不是从 1 开始）
- 连接空闲时禁用慢启动
- 启用窗口缩放（RFC1323）
- 关注在服务端修改就可以使用的算法

客户端实践：

- 尽量少发：减少 send 调用
- 缩短传输距离：就近接入
- 长连接：重用 TCP 连接
- 压缩传输数据
- 写满 TCP 发送 buffer（尤其是大数据传输，采用流式而非分块传输）
- 控制好数据处理线程

网络协议设计（数据）：

- 协议要素：语法，语义，时序
- 语法：文本编码/二进制编码
 - 文本编码：符号分割 (遇到符号转义), Key-value(http), XML 文件——> 文本编码空间浪费大
 - 二进制（数据）编码：TLV (Tag-Length-Value), Protobuf
 - 其他考虑因素：压缩，加密 (RSA,AES,DES)，编解码效率
- 语义：业务数据与信令数据
 - 扩展性：规模扩展与功能扩展
 - 兼容性：增加 required,optional 字段声明，使用版本号或自描述
 - 其他考虑因素：并发，错误处理，简洁，高效
- 时序同步：时序图，状态图

4 终端存储技术

Day 4 2025/10/8

终端的存储主要是磁盘的 IO，这是相对来说比较慢的操作，对性能的影响巨大。

用户数据分类：

- 文件：大块连续二进制数据，不关心内部内容——语音，图片，视频，文档

当文件数量很大时，采用 MD5 进行哈希离散化处理，取前两个字符作为目录（分为 256 份）

- 键值对：小型，离散数据——配置类数据
- 数据库（SQLite）：同类大量重复的数据，高效查询写入表——联系人，聊天记录（消息）

本地存储，基于单个文件，支持大部分 SQL 语句，支持事务，加密，跨平台

添加索引 userName，为数据建目录，产生额外开销

使用存储过程，一次将批量数据存储至数据库

扩展性——消息表增加可扩展字段，跨表查询

数据库设计优化：

- 使用定长数据做索引值，可减少 Page 内碎块
- 分表，把经常需要改动的数据设计为单独的表
- 使用整数做索引，避免索引是另一个符合索引的前缀
- 尽量把大的数据块设计在表的最后面

	XML	JSON	Protcol Buffers
数据形式	文本	文本	二进制
兼容性	✓	✓	✓
易读性	✓	✓	
访问速度			最快
序列化后大小			最小

图 7: 键值对存储序列化技术对比

5 Android 安全开发

Day 5 2025/10/10

安全课题：

- 黑客攻防 -> 服务器保护，主机保护，软件，web 漏洞
- 盗号产业链 -> 反作弊，防诈骗，财产与隐私保护
- 社会和谐 -> 色情，暴力，谣言

微信终端安全：

- 保护安装包的完整

微信多开 -> 使用 package 标识 app 的唯一性，修改安装包里的 package 实现多开

重打包过程：apktool 反编译 -> 代码修改 -> apktool 重打包 -> 重新签名

防御手段：proguard(代码混淆)，动态上报，有损服务
- 用户数据的保密

本地数据安全：root 权限，sd 卡权限可以读取手机上其他 app 的数据 -> 加密本地信息，敏感信息存储在云端

网络数据保密：图片是私有协议加密上传，但是 http 明文分发（成本，效率） -> 加密网络交互数据，非敏感数据可不加密
- 提高代码的安全性

拒绝服务攻击 -> 解决 NPE（输入有效验证），白名单判断（白名单外类型忽略调用，默认拒绝），禁止外部调用（最小权限原则）

慎用第三方库 -> 非开源第三方库更加危险，一旦使用第三方库注意更新

位置	数据
/data	加密存储聊天记录, 通信录等
/sdcard	禁止存储敏感数据
云端	交易信息, 银行信息等

图 8: 敏感数据存储

- 输入验证(Validate input)
 - 警示编译器警告(Heed compiler warnings)
 - 针对安全策略的架构和设计(Architect and design for security policies)
 - 保持简单(Keep it simple)
 - 默认拒绝(Default deny)
 - 最小权限原则(Adhere to the principle of least privilege)
 - 清理输出的数据(Sanitize data sent to other systems)
 - 纵深防御原则(Practice defense in depth)
 - 使用有效的质量保证技术(Use effective quality assurance techniques)
 - 统一安全编码规范(Adopt a secure coding standard)
- CERT 十大安全编程实践

图 9: 十大安全编程实践

6 多终端平台技术

Day 6 2025/10/11

iOS 开发简介

- Core OS -> Socket, Keychain Access
- Core Services -> SQLite, Threading
- Madia (多媒体)
- Cocoa Touch (交互)

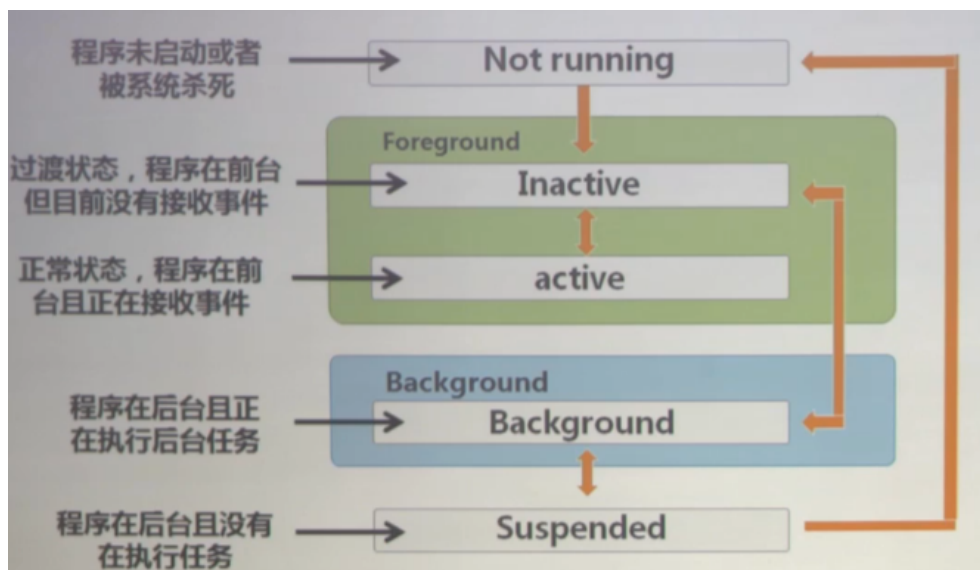


图 10: 应用状态变迁

耦合 -> MVC(Model, View, Controller)

Beta Test App(邀请 2000 个用户审核) -> Submit App(提交审核)

Push 机制

- Apple Push Notification Service (苹果 push)
- 消息格式: Json, 提醒样式: alert, badge, sound
- 新增能力: 后台运行能力, VOIP, 消息更加丰富
- 安全性非常优秀
- 长连接: 连接管理, 数据读写, 数据编解码, 任务管理

程序间通信: 微信 Open api

- iOS 如何调用第三方应用 -> UrlSchema, Universal Link (iOS9)
- iOS 程序间如何传递数据 -> PasteBoard, Schema, App Group
- 微信感知第三方 App -> 通过 canOpenUrl 方法检测每个 App (安卓)

内存管理

- Java 内存回收: 不可到达对象, 循环引用
- iOS Objective-C: 引用计数 (MRC (手动), ARC (自动)) -> 引用计数如 0 释放对象

7 iOS UI 技术

Day 7 2025/10/21

iOS APP 特点

- 流畅，现代的用户体验
- 克制的权限，隐私授权
- 唯一的分发渠道，严格的审核
- 唯一的操作系统生态

iOS UI 概述

- 人机交互准则（Human Interface Guidelines）
- UIKit – Apple 提供的兼容所有 OS 的 UI 框架
 - 官方封装的符合人机交互准则的库
 - 渲染，布局，事件

iOS UI 开发快速入门

- 只能在 macOS 开发，从创建到运行（Xcode）
- 利用 UIKit 显示基本的 App 内容
 - UIApplication 应用
 - UIWindow ，视图的根节点
 - UIViewController ，控制 window 中显示的控件
 - UIView

- 层级树（按层显示），坐标系（相对于某个原点的位置）

编程技巧

- 兼容不同的屏幕-> 布局管理思路：子 view 决定自身大小，父 view 决定子 view 的位置，基于相同坐标系运算
- 为 App 添加丰富的动画 -> UIView Animation, Core Animation, Property-Based Animation
- 利用手势于有用户交互 -> 双击, 长按, 拖拽（使用系统专门自带 class）